CICSA PROJECT

SOC DETECTION ENGINEERING LAB USING WAZUH, SHUFFLE SOAR AND TELEGRAM

SUBMITTED BY

1. Mohamed Adnan A T
2. Pranav TN

SUBMITTED TO: ALEX DAVY

COURSE: CICSA

DATE: 6 NOVEMBER 2026

ABSTRACT

Security Operations Centres (SOCs) depend on continuous monitoring, accurate detection and a fast response to suspicious activity. This project demonstrates how these three stages can be built into a small home SOC laboratory using freely available tools. A Wazuh SIEM server was installed on an Ubuntu Server virtual machine running in Oracle VirtualBox, and a Windows endpoint was enrolled as a Wazuh agent so that its security events could be collected and analysed.

Custom detection rules were written in Wazuh to identify repeated failed Windows logons, including a general brute-force rule and rules aimed at a dedicated test account named “SOC Test”. To close the loop between detection and notification, Shuffle SOAR was connected to Wazuh through a webhook and to a Telegram bot, so that every matching alert is delivered to the analyst as a Telegram message. Finally, an automated response was implemented using a custom Wazuh active-response program that disables the targeted Windows account when the detection fires.

The project documents the complete progression from monitoring, to detection, to notification and to automated response, and provides a practical reference for students who wish to build a similar detection engineering lab.

INDEX

Section	Topic
Chapter 1	**Introduction to SOC and Wazuh**
1.1	Introduction to SIEM
1.2	Features of Wazuh
1.3	Project Objectives
Chapter 2	**Lab Environment and Installation**
2.1	Hardware and Software Requirements
2.2	VirtualBox Configuration
2.3	Ubuntu Server Installation
2.4	Wazuh Installation
2.5	Windows Agent Installation and Connection
Chapter 3	**Detection Engineering**
3.1	Windows Security Event Monitoring
3.2	Custom Rule Configuration
3.3	General Brute-Force Detection
3.4	Account-Specific Detection
Chapter 4	**SOAR Integration and Alerting**
4.1	Introduction to Shuffle SOAR
4.2	Webhook Configuration
4.3	Telegram Bot Integration
4.4	Alert Notification Testing
Chapter 5	**Automated Incident Response**
5.1	Active Response in Wazuh
5.2	Custom Python Script
5.3	Executable Creation and Configuration
5.4	Account Disabling Demonstration
Chapter 6	**Results and Conclusion**
References	**Documentation and Resources**

CHAPTER 1

Introduction to SOC and Wazuh

A Security Operations Centre (SOC) is the team, process and technology set responsible for monitoring an organisation's systems, detecting security incidents and responding to them. SOC analysts spend much of their time working with alerts produced by security tooling, deciding which alerts represent real threats and acting on them. This project reproduces, on a small scale, the core SOC workflow of collecting logs, detecting suspicious behaviour, notifying an analyst and taking a response action.

1.1 Introduction to SIEM

Security Information and Event Management (SIEM) platforms collect log data from many sources such as operating systems, applications and network devices, normalise it into a common format, and analyse it against detection rules. When a rule matches, the SIEM raises an alert with a severity level so that analysts can investigate. A SIEM also keeps the collected events searchable, which supports threat hunting, auditing and incident investigation.

1.2 Features of Wazuh

Wazuh is a free and open-source security platform that combines SIEM and XDR capabilities. It consists of a manager that analyses data, an indexer that stores it, a dashboard for visualisation, and lightweight agents installed on monitored endpoints. The features relevant to this project are:

- Log data collection and analysis from endpoints, including Windows event logs.
- A rule-based detection engine with a large set of built-in rules and support for user-defined custom rules.
- Correlation of events, such as counting repeated events within a time frame.
- Threat hunting and security event dashboards, with mapping of alerts to MITRE ATT&CK.
- File integrity monitoring, configuration assessment and vulnerability detection.
- Active Response, which allows scripts or programs to run automatically on an endpoint when a rule is triggered.
- Integrations that forward alerts to external tools through webhooks.

1.3 Project Objectives

The objectives of this project were to:

- Build a home SOC laboratory using VirtualBox, Ubuntu Server and Wazuh.
- Connect a Windows endpoint to the Wazuh manager and monitor its security events.

- Write custom Wazuh rules that detect repeated failed Windows logons.
- Integrate Shuffle SOAR and a Telegram bot so that alerts are delivered as notifications.
- Implement an automated response that disables a Windows account when the detection fires.

CHAPTER 2

Lab Environment and Installation

This chapter describes the laboratory environment and the installation of the Ubuntu Server virtual machine, the Wazuh SIEM and the Windows agent.

2.1 Hardware and Software Requirements

The lab was built on a Windows laptop which also acted as the monitored endpoint. The software used is listed below.

- Oracle VirtualBox – hypervisor hosting the Wazuh server.
- Ubuntu Server (64-bit) – operating system of the Wazuh server virtual machine.
- Wazuh 4.14.9 – SIEM, comprising the indexer, manager, Filebeat and dashboard.
- Windows host with the Wazuh agent – monitored endpoint.
- Shuffle SOAR (cloud) – workflow automation.
- Telegram and a Telegram bot – notification channel.
- Python and PyInstaller – used to build the custom active-response program.

2.2 VirtualBox Configuration

A virtual machine named “Ubuntu Server” was created in VirtualBox with the type set to Ubuntu (64-bit). The virtual machine was allocated approximately 7.6 GB (7652 MB) of base memory so that the Wazuh indexer, manager and dashboard could run together on a single machine. Figure 2.1 shows the System settings of the virtual machine.

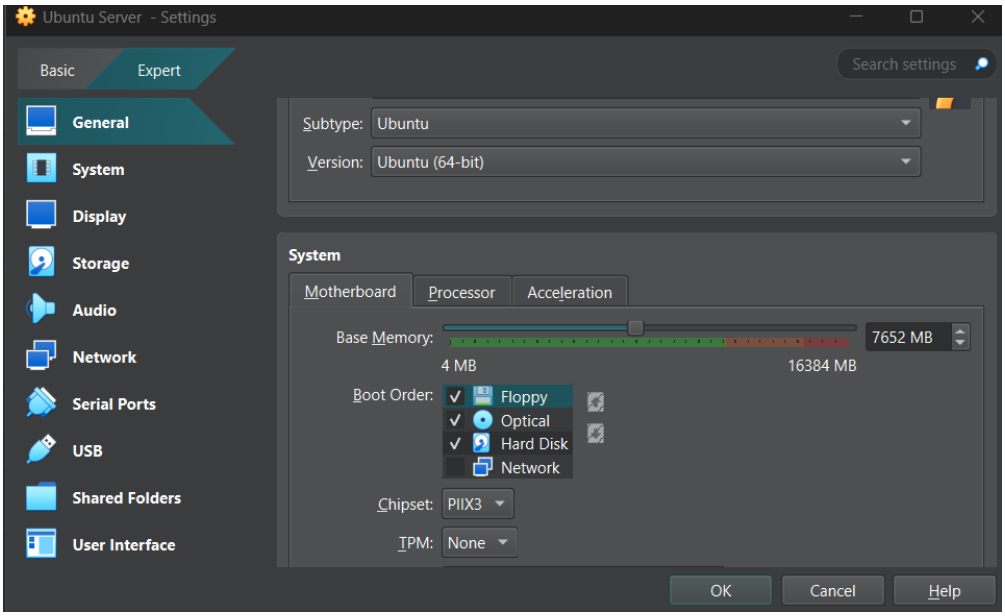


Figure 2.1: VirtualBox settings of the Ubuntu Server virtual machine

2.3 Ubuntu Server Installation

Ubuntu Server was installed in the virtual machine using the text-based Ubuntu installer. Figure 2.2 shows the installer applying the system configuration, including the keyboard, networking, file system table and boot loader steps.

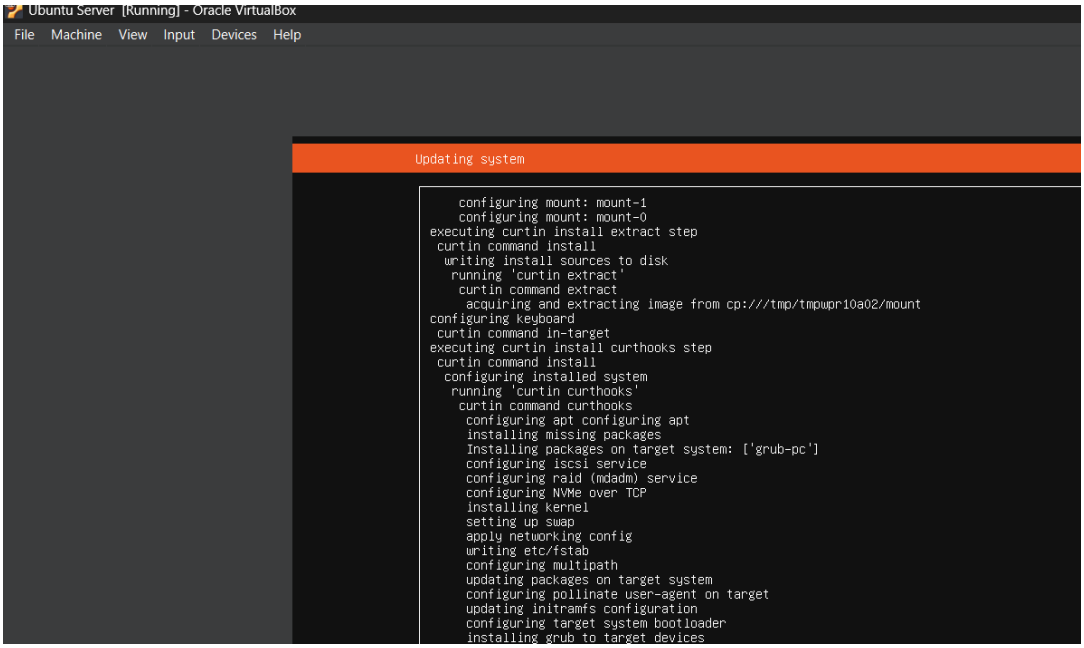


Figure 2.2: Ubuntu Server installation in progress

During the final system configuration stage the installer retrieved and installed the OpenSSH server, which allows the server to be managed remotely, and then downloaded the latest security updates, as shown in Figure 2.3.

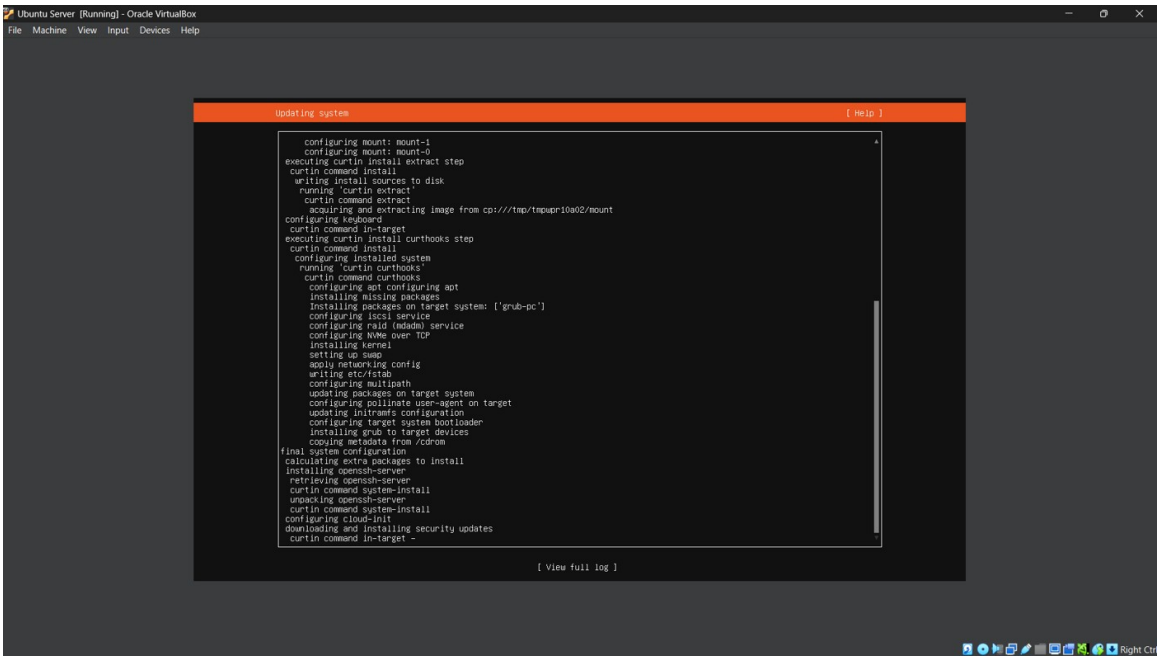


Figure 2.3: Ubuntu Server installation completing with OpenSSH server installed

2.4 Wazuh Installation

Wazuh was installed on the Ubuntu server using the official installation assistant. The assistant was downloaded with curl and executed with the -a option, which installs all components (indexer, server and dashboard) on one host. The commands used were:

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
ls -lh wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

Figure 2.4 shows the terminal output of the installation. The assistant generated the certificates, installed and started the Wazuh indexer, installed and started the Wazuh manager, and then installed Filebeat. The installation was carried out on 3 November 2026.

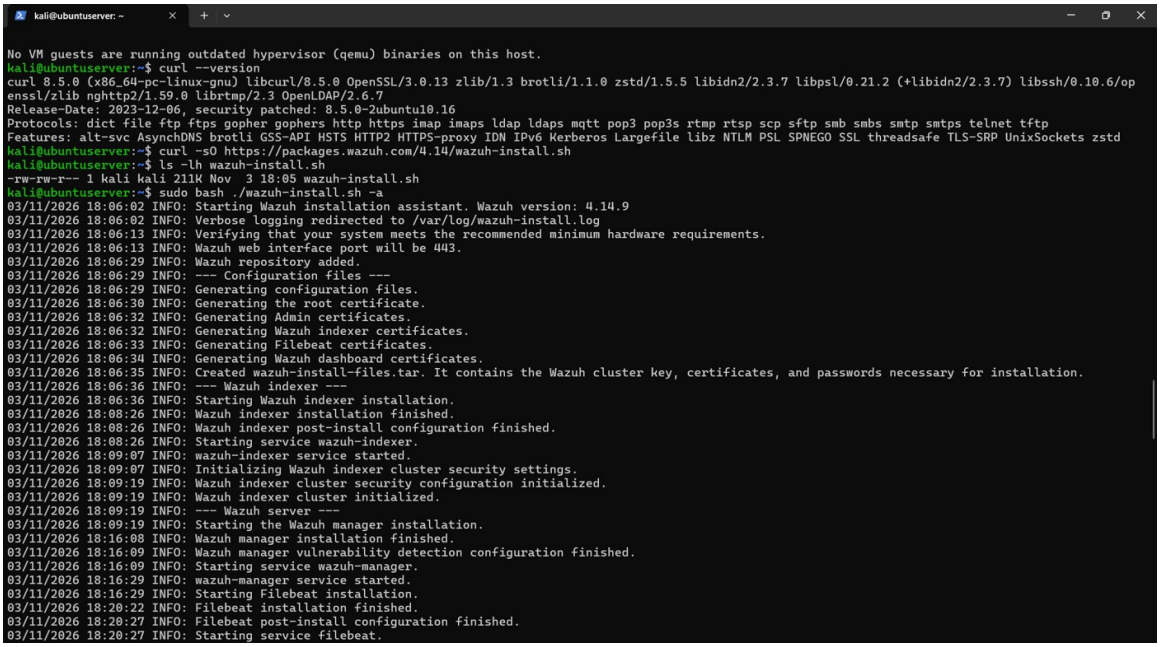


Figure 2.4: Wazuh SIEM installation output

After installation the Wazuh dashboard was opened in the browser. Figure 2.5 shows the dashboard overview. At this stage no agents were registered, which is why the Agents Summary panel asks for an agent to be deployed; the alert counters reflect events generated by the server itself.

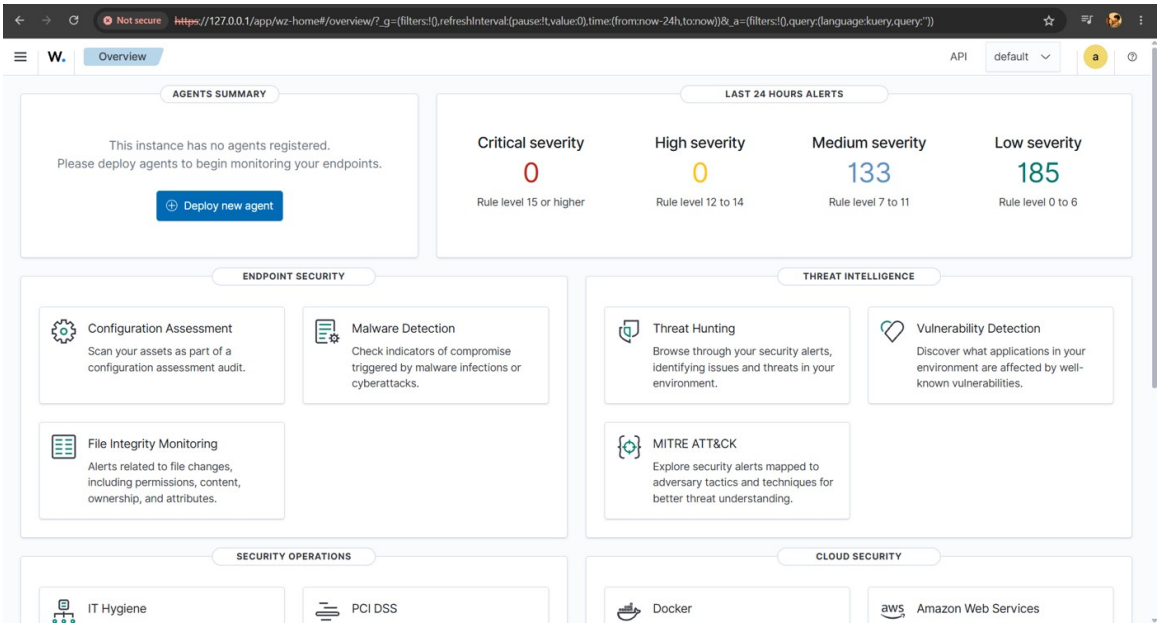


Figure 2.5: Wazuh dashboard overview before agent enrolment

2.5 Windows Agent Installation and Connection

The Windows endpoint was registered with the manager using the manage_agents utility on the Ubuntu server. A previous agent entry (ID 001) was first removed, and a new agent named “Windows-Host” was added, which was assigned ID 002. The agent key was then extracted so that it could be entered on the Windows agent. Figure 2.6 shows this procedure.

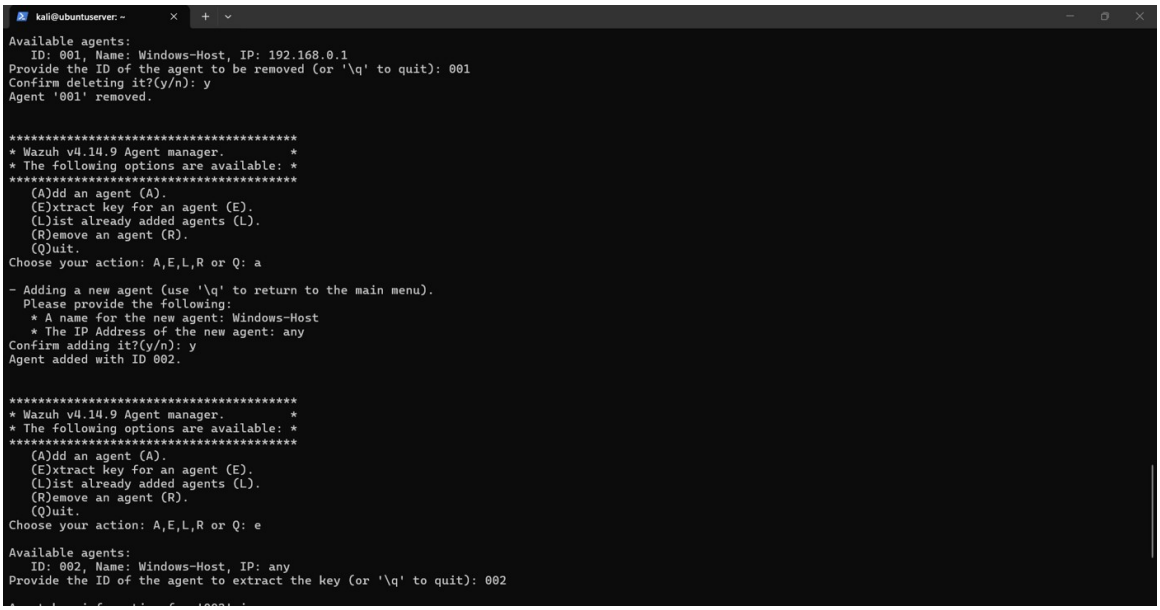


Figure 2.6: Registering the Windows agent with manage_agents

On the Windows machine the Wazuh agent was installed and configured with the manager address and the authentication key. Figure 2.7 shows the Wazuh Agent window reporting version 4.14.9, the agent “Windows-Host (002)” and the status “Running”. The authentication key has been masked in the figure.

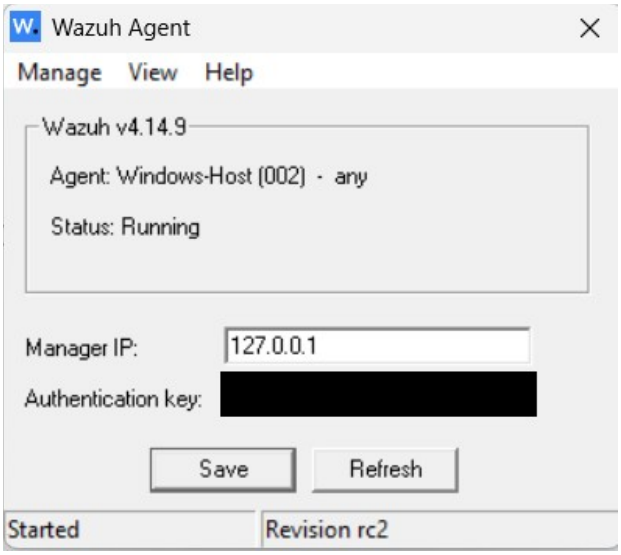


Figure 2.7: Windows agent running and connected to the Wazuh manager

CHAPTER 3

Detection Engineering

Detection engineering is the process of turning observed attacker behaviour into reliable detection logic. In this chapter the failed Windows logon events collected by the agent are examined, and custom Wazuh rules are written to detect repeated failures.

3.1 Windows Security Event Monitoring

Once the agent was connected, Windows security events began to appear in Wazuh. Failed logon attempts on the Windows host generate Windows security events that Wazuh decodes and classifies under its built-in rules. Figure 3.1 shows the Threat Hunting dashboard filtered on "Authentication failure" for the Windows-Host agent, where the failed logon appears as an authentication failure alert.

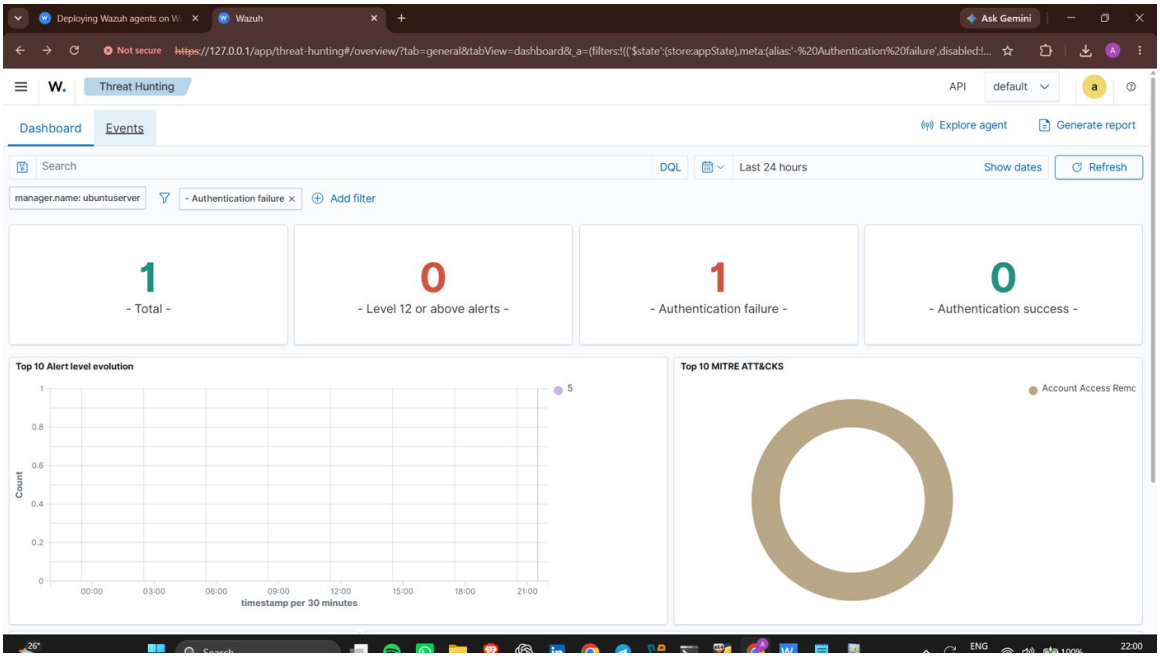


Figure 3.1: Threat Hunting dashboard showing an authentication failure

Opening the event in the Events tab shows the full document. In Figure 3.2 the event originates from agent 002 (Windows-Host) and records fields such as the logon type (2, an interactive logon), the authentication package (Negotiate), the process svchost.exe and the failure status code. The built-in rule that matches this single failed logon is rule 60122, which is the base rule used by the custom rules in the next sections.

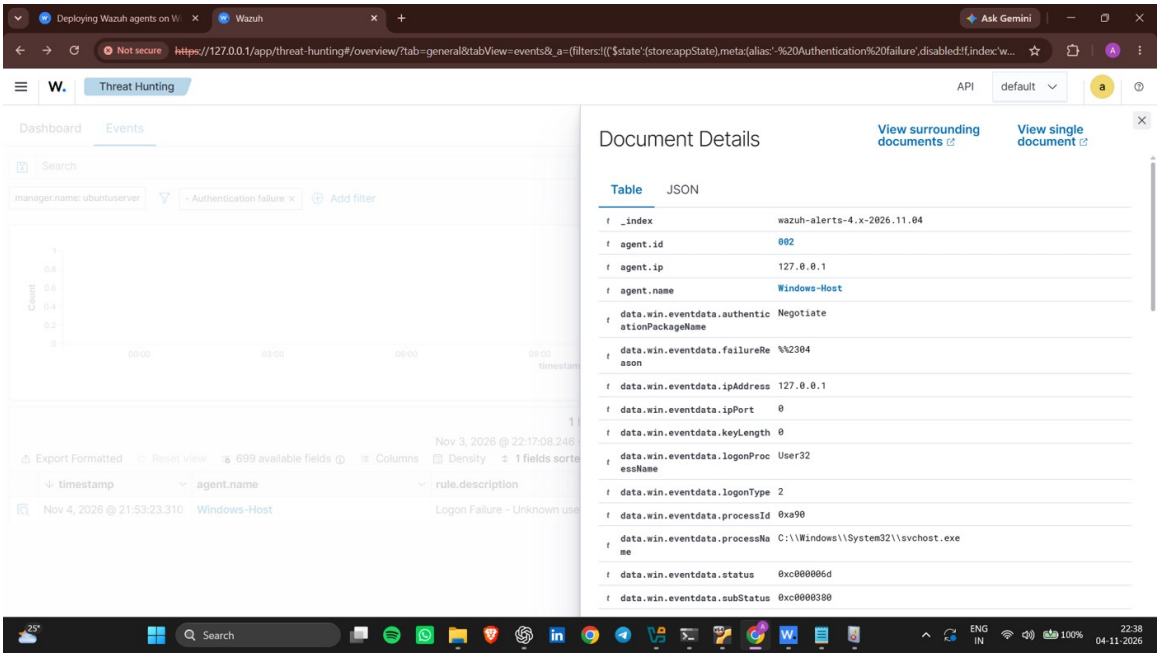


Figure 3.2: Details of a Windows logon failure event

3.2 Custom Rule Configuration

Wazuh stores user-defined rules in /var/ossec/etc/rules/local_rules.xml on the manager. Custom rule IDs start at 100000, so they never clash with the built-in rules. The file was edited on the Ubuntu server with the nano editor. Three custom rules form the detection logic of this project:

Rule ID	Purpose
100100	Detects multiple Windows logon failures within 60 seconds
100102	Identifies a failed login attempt specifically against the SOC Test account
100101	Detects three failed login attempts against SOC Test within 60 seconds

3.3 General Brute-Force Detection

Rule 100100 is the general brute-force rule. It builds on the built-in Windows logon failure rule 60122 and raises a level 10 alert when the same agent records three such failures within 60 seconds. The rule is shown in Figure 3.3 and its definition is reproduced below.

```
<rule id="100100" level="10" frequency="3" timeframe="60">
  <if_matched_sid>60122</if_matched_sid>
  <same_field>agent.id</same_field>
  <description>Multiple Windows authentication failures detected within 60
seconds.</description>
  <group>authentication_failed,brute_force,</group>
</rule>
```

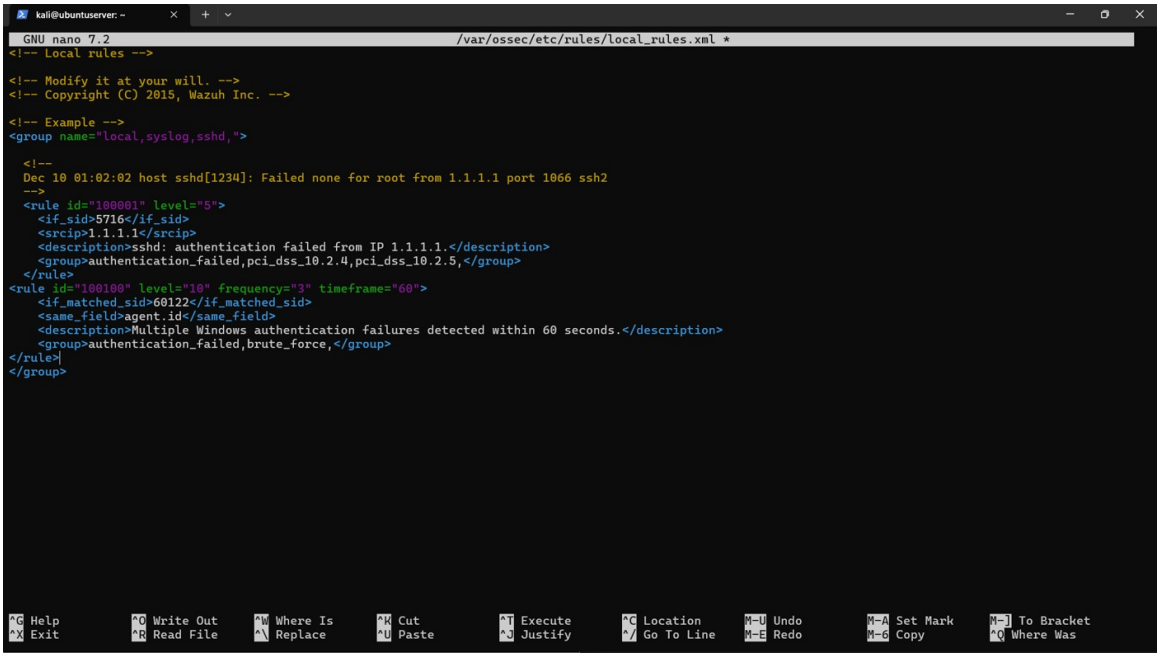


Figure 3.3: Rule 100100 in local_rules.xml

The key attributes are: frequency="3" and timeframe="60", which require three matching events within 60 seconds; if_matched_sid, which points to the base logon failure rule; and same_field agent.id, which ensures the failures come from the same endpoint. To test the rule, several incorrect passwords were entered at the Windows logon screen. Figure 3.4 shows the resulting alert in Wazuh: rule 100100, level 10, "Multiple Windows logon failures within 60 seconds", raised for Windows-Host.

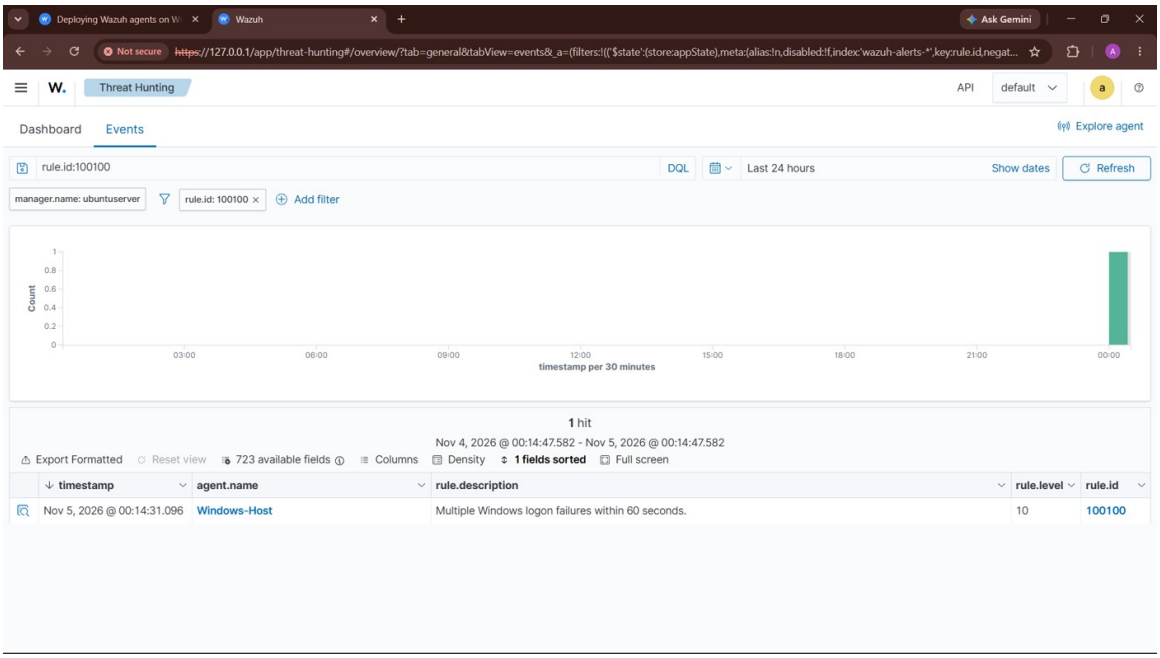


Figure 3.4: Wazuh alert for rule 100100 (repeated failed logons)

3.4 Account-Specific Detection

A second pair of rules focuses on a dedicated test account named “SOC Test”, which was created so that the response could be demonstrated safely without affecting the main user account. Rule 100102 identifies each individual failed login attempt against the SOC Test account, while rule 100101 correlates repeated failures against that account and fires when three failed attempts occur within 60 seconds. Rule 100101 is the rule that is used to trigger the automated response described in Chapter 5.

CHAPTER 4

SOAR Integration and Alerting

Detection is only useful if an analyst is told about it quickly. This chapter describes how Shuffle SOAR and a Telegram bot were used to deliver Wazuh alerts as instant notifications. The alert flow is:

Wazuh → Webhook → Shuffle SOAR → Telegram Bot → Security Notification

4.1 Introduction to Shuffle SOAR

Shuffle is an open-source Security Orchestration, Automation and Response (SOAR) platform. Automations are built as workflows made of triggers and apps connected on a canvas, for example a webhook that receives an alert followed by an app that sends a message. A new workflow was created on the Shuffle web platform, as shown in Figure 4.1.

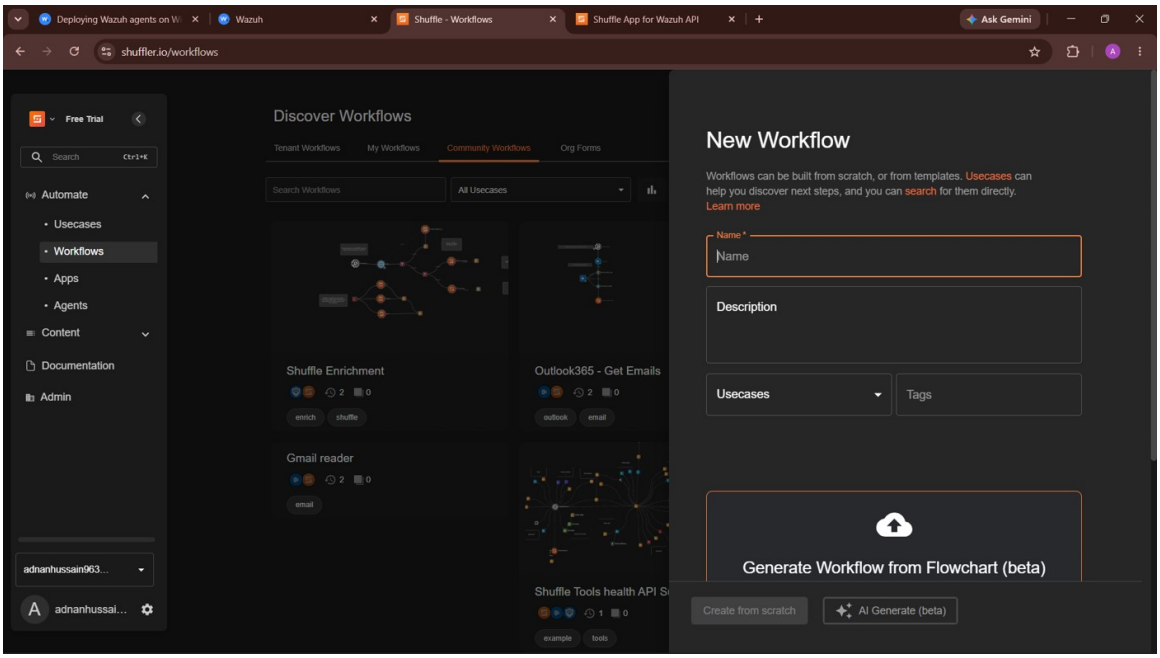


Figure 4.1: Creating a new workflow in Shuffle

4.2 Webhook Configuration

The workflow, named “Wazuh Brute Force Alert.”, starts with a Webhook trigger. The webhook provides a URL that Wazuh can post alerts to. When Wazuh raises an alert for the configured rule, the alert data is sent to the webhook and the workflow starts. The webhook address is private and is therefore not shown in this report.

4.3 Telegram Bot Integration

A Telegram bot, “Wazuh SOC Alert Bot”, was created for the notifications, and the Telegram Bot app in Shuffle was connected to the webhook. The app uses the “Send message” action, whose body is a JSON message containing the destination chat and the alert text: a fixed

heading, the rule ID, the severity, the description, and the agent name and ID. Figure 4.2 shows the workflow, with the Webhook node connected to the Telegram Bot node, and the configuration of the Send message action. The bot token and chat identifier have been masked.

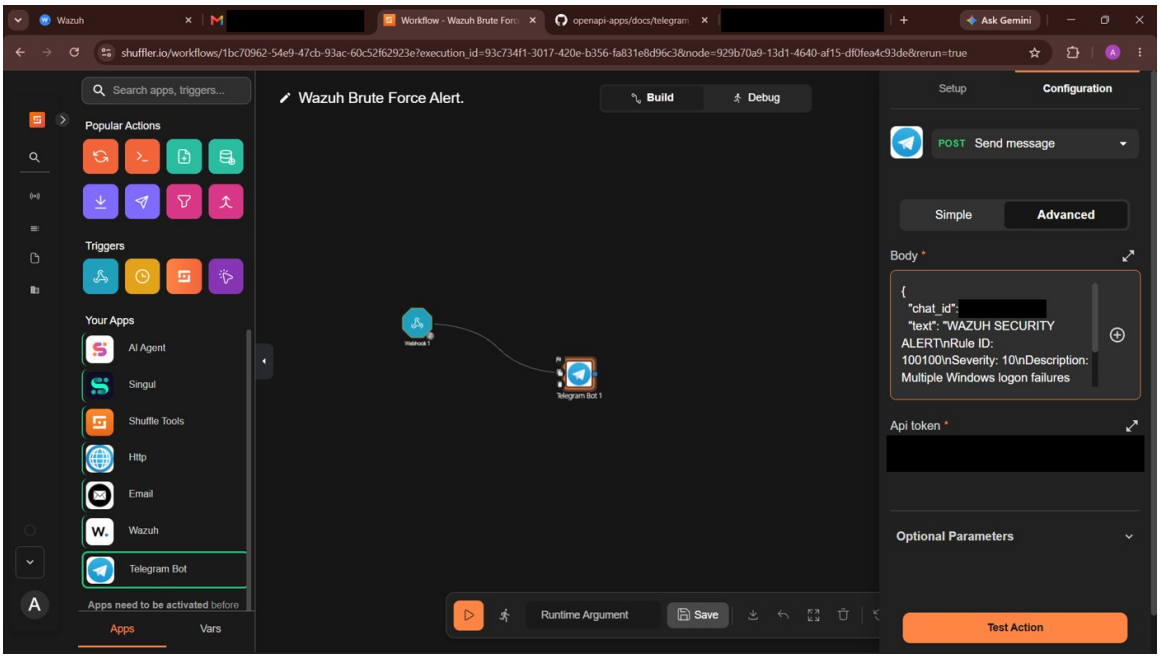


Figure 4.2: Shuffle SOAR workflow with Webhook and Telegram Bot nodes

4.4 Alert Notification Testing

The workflow was tested by first sending a test notification and then by generating failed logons on the Windows host. Figure 4.3 shows the conversation with the bot. The test message “Test notification successful!” was received first, followed by several “WAZUH SECURITY ALERT” messages for Rule ID 100100 with severity 10, the description “Multiple Windows logon failures within 60 seconds.”, and the agent Windows-Host (ID 002). This confirms that the complete chain from Wazuh to Telegram works.

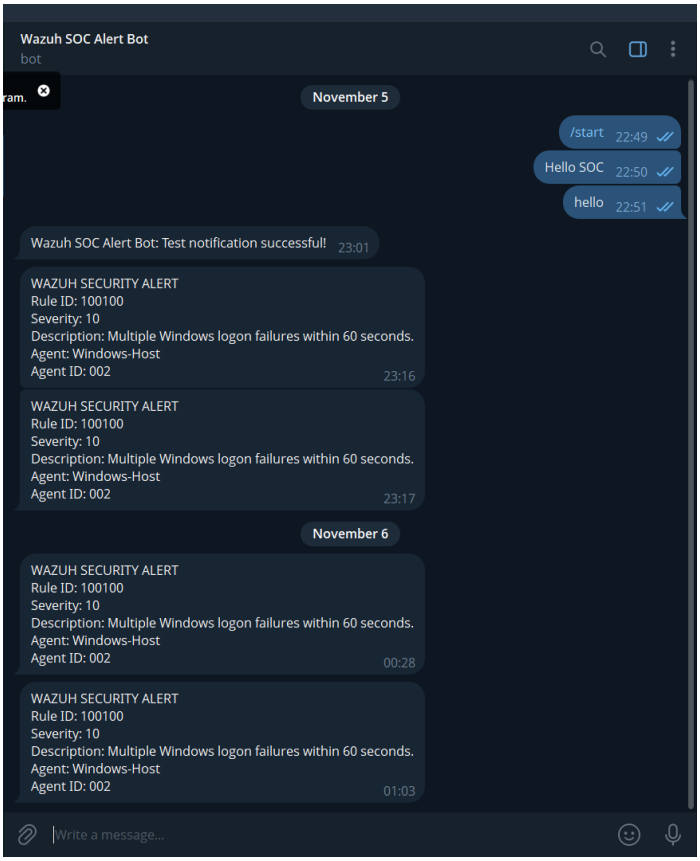


Figure 4.3: Security alert notifications received in Telegram

CHAPTER 5

Automated Incident Response

The final stage of the lab is an automated response. When the detection rule fires, Wazuh instructs the Windows agent to disable the SOC Test account, which stops the suspected brute-force attack from continuing against that account.

5.1 Active Response in Wazuh

Active Response is the Wazuh capability that runs a script or program on an endpoint when a rule, rule group or alert level is triggered. A trigger starts the program through a defined command, and an Active Response configuration in ossec.conf determines when and where the command runs. Wazuh passes the full alert to the program as a JSON message on standard input, and the program extracts the information it needs. Figure 5.1 shows the Wazuh documentation page on custom active response scripts that was used as the reference for this work.

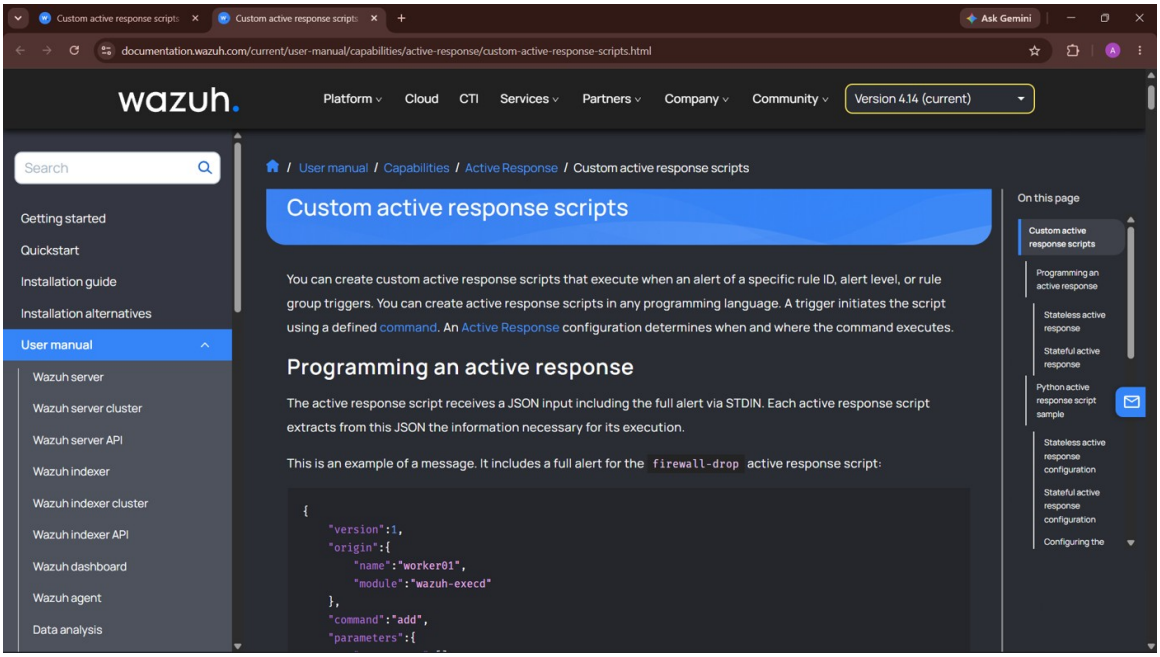


Figure 5.1: Wazuh documentation on custom active response scripts

5.2 Custom Python Script

A custom program named disable-soc-test was written in Python. When it is executed by Wazuh it disables the local Windows account “SOC Test” and writes the outcome to the agent’s active-responses.log file. The logged success message is “disable-soc-test: SUCCESS: SOC Test account disabled.”, which is shown in Section 5.4.

5.3 Executable Creation and Configuration

The Wazuh agent on Windows runs executables from its active-response\bin folder, which by default contains programs such as restart-wazuh.exe and route-null.exe. The Python script was therefore converted into a Windows executable with PyInstaller. PyInstaller was installed with pip, the executable was copied into the active-response\bin folder of the agent, its presence was confirmed with Test-Path, and the Wazuh agent service was restarted so that the change took effect:

```
python -m pip install pyinstaller
Copy-Item "$env:USERPROFILE\dist\disable-soc-test.exe" "C:\Program Files
(x86)\ossec-agent\active-response\bin\disable-soc-test.exe"
Test-Path "C:\Program Files (x86)\ossec-agent\active-response\bin\disable-
soc-test.exe"
Restart-Service -Name WazuhSvc
```

Figure 5.2 shows these steps in an administrator PowerShell window. Test-Path returned True, confirming that the executable is deployed on the agent. PyInstaller also warns that it should not be run from C:\Windows\system32 or as administrator, so the build was run from the folder that contains the script.

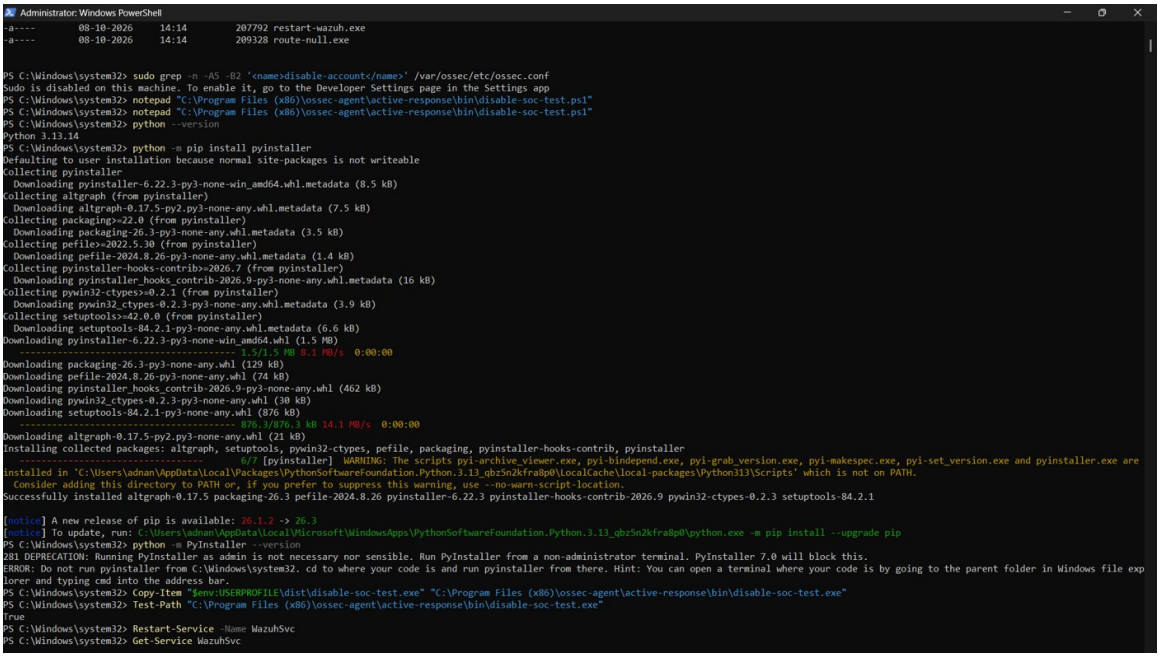


Figure 5.2: Installing PyInstaller and deploying the executable to the Windows agent

5.4 Account Disabling Demonstration

To test the response, failed logons were generated against the SOC Test account. The detection fired and Wazuh executed the program on the agent. The result was verified in PowerShell with the following commands:

```
Get-LocalUser -Name "SOC Test" | Select-Object Name, Enabled
```

```
Get-Content "C:\Program Files (x86)\ossec-agent\active-response\active-
responses.log" -Tail 30
```

In Figure 5.3 the Enabled property of SOC Test is False, and the active-responses.log file contains the entries “disable-soc-test: SUCCESS: SOC Test account disabled.” on 6 November 2026 at 20:45:17 and again at 20:59:58, corresponding to two separate test runs. Between the runs the account was re-enabled by hand with Enable-LocalUser so the test could be repeated.

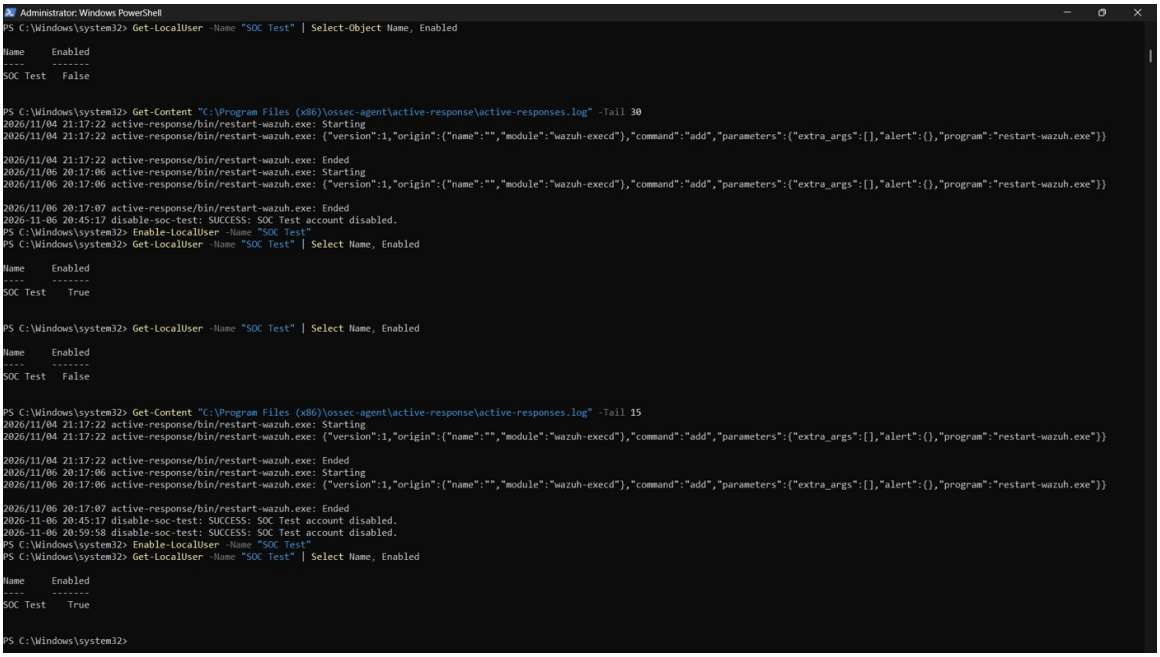


Figure 5.3: Verification that the SOC Test account was disabled

Finally, the Windows logon screen was checked. Figure 5.4 shows the SOC Test account displaying the message “Your account has been disabled. Please see your system administrator.” This is the visible proof that the automated response worked.

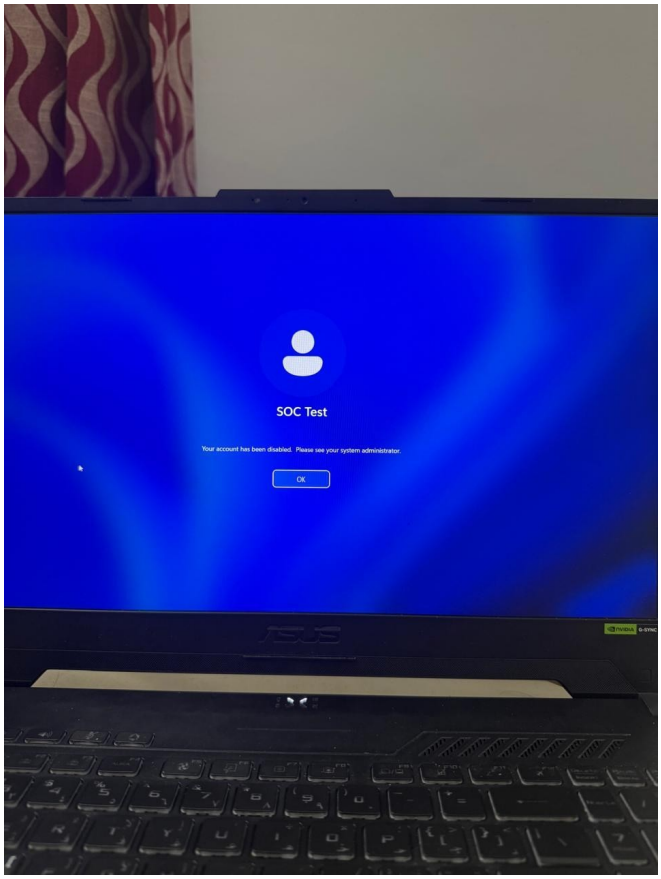


Figure 5.4: Windows login screen showing the disabled SOC Test account

CHAPTER 6

Results and Conclusion

Results

- The Ubuntu Server virtual machine and the Wazuh SIEM were installed and the dashboard was accessible (Figures 2.1 to 2.5).
- The Windows endpoint was enrolled as agent 002 and reported the status Running (Figures 2.6 and 2.7).
- Failed Windows logons were collected and shown in Wazuh, and the custom rule 100100 successfully detected repeated failures within 60 seconds (Figures 3.1 to 3.4).
- Account-specific detection of the SOC Test account was implemented with rules 100102 and 100101 (Section 3.4).
- Shuffle SOAR delivered Wazuh alerts to Telegram through the webhook and the Telegram bot (Figures 4.2 and 4.3).
- The SOC Test account was automatically disabled by the custom active-response program, as shown by the PowerShell output, the active-responses.log entries and the Windows login screen (Figures 5.3 and 5.6).

Conclusion

This project built a working home SOC laboratory that covers the full detection and response cycle. Wazuh provided the monitoring and detection, custom rules turned repeated failed logons into meaningful alerts, Shuffle SOAR and Telegram delivered those alerts to the analyst in real time, and a custom active-response executable automatically disabled the targeted account. Together these components show how a SIEM and a SOAR platform can reduce the time between an attack and a response.

The lab also gave practical experience with virtualisation, Linux server administration, XML rule writing, API-based integrations and Windows administration, all of which are relevant to the work of a SOC analyst. As future improvements, the detection could be extended to other attack types, the response could be limited to specific source addresses or time windows, and further SOAR steps such as ticket creation could be added.

REFERENCES

1. Wazuh Documentation. https://documentation.wazuh.com

2. Wazuh Active Response Documentation – Custom active response scripts. https://documentation.wazuh.com/current/user-manual/capabilities/active-response/custom-active-response-scripts.html

3. Shuffle SOAR Documentation. https://shuffler.io/docs

4. Telegram Bot API. https://core.telegram.org/bots/api

5. Ubuntu Server Documentation. https://ubuntu.com/server/docs